

Связи между таблицами: зачем и как

Зачем разделять данные на таблицы и как проектировать связи между ними

В теме про FOREIGN KEY мы разобрали **синтаксис** внешнего ключа. Здесь разберёмся, **зачем** вообще делить данные на несколько таблиц и **как по шагам** спроектировать связи между ними. Пример — учебная часть колледжа: группы, студенты, курсы и записи на курсы. На этих же таблицах построены практические задания.

Проблема: всё в одной таблице

Допустим, учебная часть ведёт всё в одной большой таблице — как в Excel:

student	group_name	curator	course	teacher
Алишер	WEB-21	Каримова	Python	Иванов
Алишер	WEB-21	Каримова	Базы данных	Петрова
Мадина	WEB-21	Каримова	JavaScript	Сидоров
Тимур	IB-22	Юсупов	Python	Иванов
Тимур	IB-22	Юсупов	Базы данных	Петрова

Выглядит просто, но у такой таблицы четыре серьёзные проблемы.

- 1. Дублирование (избыточность).** Куратор «Каримова» записан столько раз, сколько у студентов группы WEB-21 записей на курсы. Преподаватель «Иванов» повторяется в каждой строке с курсом Python. Данных становится в разы больше, чем нужно.
 - 2. Аномалия обновления.** Куратором WEB-21 назначили другого человека — придётся менять фамилию **во всех** строках группы. Пропустили одну строку, и в базе у одной группы стало два разных куратора. Какому верить?
 - 3. Аномалия вставки.** Открыли новый курс «Алгоритмы», но на него ещё никто не записался. Добавить его нельзя: в каждой строке обязательно должен быть студент. Курс как бы «не существует», пока нет студентов.
 - 4. Аномалия удаления.** Мадина отчислилась, удаляем её строку — и вместе с ней из базы пропадает информация, что курс JavaScript ведёт Сидоров. Он больше нигде не записан.
- Все четыре проблемы возникают по одной причине: **в одной таблице смешаны разные сущности** — студенты, группы, курсы и факт «студент учится на курсе».

Решение: одна сущность — одна таблица

Главное правило проектирования: **каждый факт хранится ровно в одном месте**. Каждая сущность получает свою таблицу со своим первичным ключом, а таблицы ссылаются друг на друга по `id`.

После разделения получается такая схема. Стрелка идёт от внешнего ключа к первичному ключу, на который он ссылается; «1» и «∞» показывают, сколько строк с каждой стороны:

```
groups (id, name, curator)
  ▲ 1
  | ∞
students (id, name, group_id → groups.id)
  ▲ 1
  | ∞
enrollments (student_id → students.id, course_id → courses.id)
  | ∞
  ▼ 1
courses (id, title, teacher)
```

И те же данные, но уже без повторов:

groups.id	name	curator
1	WEB-21	Каримова
2	IB-22	Юсупов

students.id	name	group_id
1	Алишер	1
2	Мадина	1
3	Тимур	2

courses.id	title	teacher
1	Python	Иванов
2	Базы данных	Петрова
3	JavaScript	Сидоров

student_id	course_id
1	1
1	2
2	3
3	1
3	2

Теперь:

- куратор или название группы записаны **один раз** — меняем в одной строке;
- курс можно добавить, даже если на него никто не записан;
- отчисление студента удаляет только его самого и его записи на курсы, а курсы остаются.

Как определить тип связи: два вопроса

Для каждой пары сущностей задайте два вопроса — «сколько В у одного А?» и «сколько А у одного В?»:

Пара	Сколько у одного...	Ответ	Тип связи
группа — студент	студентов в группе? / групп у студента?	много / одна	один ко многим
студент — курс	курсов у студента? / студентов на курсе?	много / много	многие ко многим
студент — студенческий билет	билетов у студента? / студентов у билета?	один / один	один к одному

От ответа зависит, **куда** ставить внешний ключ.

Один ко многим: внешний ключ — у «многих»

Ссылку хранит та сторона, которой **много**: у каждого студента одна группа, поэтому столбец `group_id` появляется в `students`.

```
CREATE TABLE groups (  
  id INTEGER PRIMARY KEY,  
  name TEXT NOT NULL UNIQUE  
);  
  
CREATE TABLE students (  
  id INTEGER PRIMARY KEY,  
  name TEXT NOT NULL,  
  group_id INTEGER NOT NULL REFERENCES groups(id)  
);
```

Почему не наоборот? Если бы список студентов хранился в группе, в одной ячейке пришлось бы писать `'1,2,5'`. По такой строке нельзя нормально искать, её нельзя проверить внешним ключом, а добавить студента можно только переписав всю строку. **Одна ячейка — одно значение.**

Многие ко многим: таблица-связка

Студент — курс: «много» с обеих сторон. Столбец `course_id` в `students` не подойдёт (у студента несколько курсов), а `student_id` в `courses` — тоже (на курсе несколько студентов). Поэтому создаём **отдельную таблицу-связку**. Каждая её строка — один факт «этот студент записан на этот курс».

```
CREATE TABLE courses (  
  id INTEGER PRIMARY KEY,  
  title TEXT NOT NULL UNIQUE  
);  
  
CREATE TABLE enrollments (  
  student_id INTEGER NOT NULL REFERENCES students(id),  
  course_id INTEGER NOT NULL REFERENCES courses(id),  
  PRIMARY KEY (student_id, course_id)  
);
```

Составной первичный ключ `(student_id, course_id)` не даёт записать одного студента на один курс дважды. Связь «многие ко многим» всегда раскладывается на две связи «один ко многим»: студент → записи и курс → записи.

В таблицу-связку можно добавлять свойства самого факта записи, например дату записи или итоговую оценку: `enrolled_at TEXT`, `grade INTEGER`.

Один к одному: внешний ключ с UNIQUE

Студенческий билет есть ровно у одного студента. Делаем обычный внешний ключ и запрещаем повторы через `UNIQUE`:

```
CREATE TABLE student_cards (  
  id INTEGER PRIMARY KEY,  
  student_id INTEGER NOT NULL UNIQUE REFERENCES students(id),  
  card_number TEXT NOT NULL,  
  issued_at TEXT NOT NULL  
);
```

Такая связь нужна редко. Обычно её используют, чтобы вынести редко используемые или «тяжёлые» данные в отдельную таблицу.

Заполнение связанных таблиц

Порядок важен: **сначала родители, потом дети**. Нельзя записать студента в группу, которой ещё нет.

```

INSERT INTO groups (id, name) VALUES (1, 'WEB-21'), (2, 'IB-22');

INSERT INTO students (id, name, group_id) VALUES
  (1, 'Алишер', 1),
  (2, 'Мадина', 1),
  (3, 'Тимур', 2);

INSERT INTO courses (id, title) VALUES (1, 'Python'), (2, 'Базы данных'), (3,
'JavaScript');

-- Алишер и Тимур записаны на Python и Базы данных, Мадина — на JavaScript
INSERT INTO enrollments (student_id, course_id) VALUES (1, 1), (1, 2), (2, 3), (3, 1),
(3, 2);

```

В связях хранятся **идентификаторы**, а не имена: (1, 2), а не ('Алишер', 'Базы данных'). Когда проверка внешних ключей включена (PRAGMA foreign_keys = ON), запись с несуществующим id отклоняется:

```

INSERT INTO enrollments (student_id, course_id) VALUES (1, 99);
-- Error: FOREIGN KEY constraint failed — купца с id = 99 нет

```

Как достать связанные данные (пока без JOIN)

Разделённые данные нужно уметь собирать обратно. Полноценный инструмент для этого — **JOIN** (следующая тема). Но многое можно сделать уже сейчас **вложенным запросом**: внутренний запрос находит id, внешний использует его.

```

-- Студенты группы WEB-21
SELECT name FROM students
WHERE group_id = (SELECT id FROM groups WHERE name = 'WEB-21');

```

Если внутренний запрос возвращает **несколько** значений, вместо = используем IN:

```

-- Кто записан на курс Python: название → id курса → id студентов → имена
SELECT name FROM students
WHERE id IN (
  SELECT student_id FROM enrollments
  WHERE course_id = (SELECT id FROM courses WHERE title = 'Python')
);

```

```

-- Сколько студентов записано на каждый курс (группировка по таблице-связке)
SELECT course_id, COUNT(*) AS students_count
FROM enrollments
GROUP BY course_id;

```

В последнем запросе видны только номера курсов. Чтобы рядом с числом стояло название курса, нужно **соединить** две таблицы в одном результате. Именно это делает JOIN.

Алгоритм проектирования

1. Выпишите **сущности** — существительные из задачи: группа, студент, курс.
2. Для каждой сущности создайте **таблицу** с первичным ключом `id` и её собственными свойствами.
3. Для каждой пары связанных сущностей задайте **два вопроса** и определите тип связи.
4. **Один ко многим** — внешний ключ в таблице «многих». **Многие ко многим** — таблица-связка с двумя внешними ключами. **Один к одному** — внешний ключ с `UNIQUE`.
5. Проверьте: нет ли данных, которые записаны **больше одного раза**? Если есть — им нужна своя таблица.

Частые ошибки

- **Список в ячейке:** `courses = 'Python, JavaScript'`. Так нельзя ни искать, ни проверять связи. Нужна таблица-связка.
- **Внешний ключ не с той стороны:** `students_id` в таблице групп вместо `group_id` в таблице студентов.
- **Копирование имён вместо ссылок:** столбец `group_name` в `students` вместо `group_id`. Это возвращает все аномалии обновления.
- **Таблица-связка без составного ключа:** тогда одного студента можно случайно записать на курс дважды.
- **Неправильный порядок вставки:** сначала дети, потом родители — внешний ключ не даст это сделать.